

Agent Control Plane: An External Stateful Control Layer for Autonomous Agents

Ryan

September 20, 2026

Abstract

Long-running autonomous agents exhibit failure modes that emerge from trajectory shape over time rather than from any single event: convergence stalls, budget exhaustion, repeated identical failures, and destructive writes that produce no error signal. The agent itself cannot reliably recognize these states because its observation window is limited to the current task context. We present Agent Control Plane, an external control layer that consumes the agent’s event stream, materializes cross-turn state, and intervenes through a deterministic kernel with probabilistic advisory input. We evaluate the system on six synthetic failure scenarios and three distribution-shift profiles. On a held-out set (300 trials, frozen configuration), the system achieves 100% detection and 0% false-positive rate on the scenarios used for development. Robustness testing across four dimensions (agent heterogeneity, task distribution shift, controller ablation, and unseen failure modes) reveals specific boundaries: the system cannot detect world-state corruption that produces no execution-telemetry signal, and the probabilistic controller systematically disagrees with deterministic rules on some intervention types. We characterize these as architectural limitations rather than implementation defects, and identify state-contract verification as the necessary next layer.

1 Introduction

An agent tasked with fixing a bug reads the code, edits a file, runs the tests, and observes the result. If the test still fails, it tries again. If it fails again, it tries again. At what point should this stop?

From inside the agent, the answer is unclear. Each tool result arrives in isolation: another failure, another signal to adjust and retry. The agent has no reliable mechanism to distinguish “I made progress and the next attempt will likely succeed” from “I have been making the same mistake for forty turns and will continue making it.”

This distinction is temporal. It requires observing the shape of the trajectory across turns, not the content of any single event. No amount of prompt engineering addresses it, because the agent’s observation window is structurally limited to what fits in its context window.

We propose that this class of failures requires an external control layer: a system that observes the agent’s event stream, maintains state across the entire execution, and intervenes when the trajectory indicates the agent is not converging, is consuming excessive resources, is performing dangerous operations, or is violating task boundaries. This layer is not a monitor (it does not merely report) and not a framework (it does not direct the agent’s next step). It decides whether the agent should continue.

2 Architecture

2.1 Overview

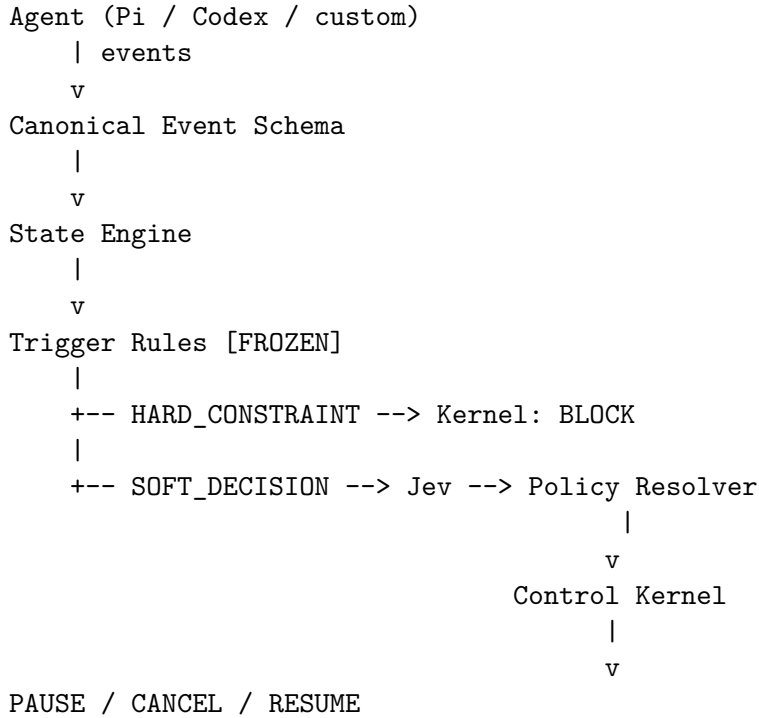


Figure 1: Control plane architecture.

2.2 Canonical Event Schema

Different agent runtimes emit events with different schemas and tool naming conventions. Pi uses **bash**, Codex-style agents use **shell** or **execute**, and custom agents may use **terminal**. Rather than hard-coding these names into the trigger logic, we define a capability-based normalization layer that maps tool names to a canonical set:

Capability	Tool names
file_read	read, cat, view
file_write	write, create
file_edit	edit, patch, modify
shell_execution	bash, shell, execute, terminal, run
search	grep, find

Table 1: Tool name to capability mapping.

This allows the State Engine and trigger rules to operate on capabilities rather than implementation-specific tool names. Adding support for a new agent runtime requires only extending the mapping table, not modifying downstream logic.

2.3 State Engine

The State Engine consumes canonical events and materializes an **AgentState** object that captures the agent’s current condition:

- **Task context:** goal, status, turn count
- **Execution:** total tool calls, total errors, recent tool call window (last 50)
- **Resources:** token usage, budget, cost
- **Files:** set of modified files, consecutive unverified writes
- **Safety:** dangerous command log, scope violations
- **Temporal:** elapsed time, recovery signal, error trend

Two properties deserve explanation. The *recovery signal* is true when the sliding window contains errors but ends with three or more consecutive successes. This distinguishes a temporary error spike followed by correction from a sustained failure loop. The *error trend* compares the error rate in the first half of the window against the second half; a rising trend without recovery indicates divergence.

2.4 Trigger Rules

Eight trigger rules operate on the materialized state. Each returns a typed decision (`should_continue` or `request_permission`) and a human-readable reason. Thresholds were tuned on a development set (seed 42, 30 trials per scenario) and frozen before held-out evaluation (seed 137, 50 trials per scenario).

Signal	Threshold	Authority	Action
Scope violation	any	HARD	BLOCK
Dangerous command	regex match	HARD	BLOCK
Silent corruption	≥ 10 unverified writes	HARD	BLOCK
Convergence stall	$> 600s$, > 5 turns	SOFT	PAUSE
Error rate	$> 40\%$, ≥ 8 calls, no recovery	SOFT	PAUSE
Error trend rising	$> 50\%$, rising, no recovery	SOFT	PAUSE
Passive stall	12 consecutive reads, 0 writes	SOFT	PAUSE
Retry count	≥ 3	SOFT	PAUSE
Token budget	$> 80\%$	SOFT	PAUSE

Table 2: Trigger rules with decision authority.

2.5 Decision Authority Hierarchy

We formalize a two-level authority model:

1. **Hard constraints:** dangerous commands, scope violations, and silent-corruption risks. These fire deterministically and cannot be overridden by the probabilistic controller. If a rule fires BLOCK and Jev returns CONTINUE, the kernel still executes BLOCK.
2. **Soft decisions:** convergence, resource, and retry-based interventions. The probabilistic controller (Jev) evaluates the compiled state and may confirm, override, or reject the rule’s recommendation. A confidence threshold of 0.5 gates Jev’s participation; below this threshold, the rule’s decision stands.

This separation prevents a probabilistic model from overriding a deterministic safety guard while still allowing nuanced judgment on non-safety decisions.

2.6 Control Kernel

The kernel is the sole executor. It receives a decision object containing the rule action, the Jev action (if any), the authority level, and a reason. It maps the final action to a concrete intervention:

- **CONTINUE**: no action
- **PAUSE**: send `abort` to the agent runtime
- **CANCEL**: send `abort` and terminate
- **ESCALATE**: send `abort` and notify the user
- **REQUEST_HUMAN_APPROVAL**: send `abort`, wait for user

3 Evaluation

3.1 Experimental setup

We evaluate on synthetic event streams generated from six scenario templates: `normal`, `transient_failure`, `infinite_loop`, `budget_bleed`, `dangerous`, and `stall`. Scenario generators use randomized parameters (number of turns, error rates, token consumption) with a controlled seed. We use a development set (seed 42) for threshold tuning and a held-out set (seed 137) for final evaluation. The controller configuration is frozen after development-set tuning.

3.2 Held-out benchmark

Scenario	N	Detection	FP Rate	Latency
normal	50	—	0.0%	—
transient_failure	50	—	0.0%	—
infinite_loop	50	100%	—	8.0
budget_bleed	50	100%	—	11.7
dangerous	50	100%	—	5.0
stall	50	100%	—	4.0
Overall	300	100%	0%	4–12

Table 3: Held-out evaluation (seed 137, 50 trials per scenario, frozen config).

These results demonstrate that the system correctly identifies all six failure scenarios while remaining silent on normal execution. However, we note several reservations about interpreting these numbers.

3.3 Interpreting the results with appropriate caution

1. **Synthetic data.** All scenarios are generated from templates we wrote ourselves. The event distributions may not reflect real agent trajectories. The 100% detection rate should be read as “the system detects what we designed it to detect,” not as a general claim.
2. **Scenario definitions overlap with trigger design.** We wrote both the scenarios and the triggers. The danger of circular validation is real: a trigger designed to detect infinite loops will detect our infinite-loop generator. Real trajectories may exhibit patterns that do not match our templates.

3. **Small scenario count.** Six scenarios is a narrow coverage claim. Real failure modes are diverse and may not fit neatly into these categories.
4. **No real-agent validation at scale.** Our end-to-end test used Pi on a single bug-fix task. We have not validated the system against a diverse set of real coding tasks at scale.

3.4 Ablation: Watchdog vs State Engine

To test whether the State Engine provides capability beyond simple event-level rules, we compare against a baseline watchdog that counts events without materializing cross-turn state:

Scenario	Detection		Context (tokens)	
	Watchdog	State Engine	Watchdog	State Engine
budget_bleed	20%	100%	2,322	409
dangerous	75%	100%	522	434
infinite_loop	0%	100%	6,084	409
stall	0%	100%	1,442	409

Table 4: Ablation results on held-out set.

The watchdog achieves 0% detection on infinite loops and stalls because these failure modes require cross-turn memory. A counter-based approach cannot distinguish between “brief error spike followed by recovery” and “sustained non-convergent failure.” The State Engine’s sliding window, recovery signal, and temporal tracking address this gap. The context-size difference (409 vs. 6,084 tokens in the worst case) reflects the MSS compilation: the State Engine maintains a fixed-size structured state regardless of trajectory length.

We note that the ablation comparison is somewhat favorable by construction: the watchdog baseline is a reasonable but not exhaustive implementation of event-level monitoring. A more sophisticated baseline (e.g., one that tracks rolling averages or emits periodic summaries) might narrow the gap on some scenarios. We did not implement such a baseline.

3.5 Robustness evaluation

We attack the architecture along four dimensions to characterize where it fails:

3.5.1 Agent heterogeneity

Three event schemas (Pi, Codex-style, custom) are normalized through the canonical layer. The canonical mapping resolves tool names like `execute` and `terminal` to `shell_execution`. Before this layer was added, dangerous command detection on custom agents was 0% because the State Engine hard-coded the tool name `bash`. After: 100% across all six agent-scenario combinations.

3.5.2 Task distribution shift

Three unseen normal task profiles test whether the system remains silent on realistic task variations:

Profile	FP Rate	Status
heavy_write	0.0%	Pass
error_recovery	0.0%	Pass
mixed_signal	0.0%	Pass

Table 5: Distribution shift: unseen normal profiles.

The error-recovery profile initially produced 60% false positives because the error-rate trigger could not distinguish “temporarily failing but recovering” from “systematically diverging.” The *recovery signal* property was added to address this: if the tail of the sliding window shows ≥ 3 consecutive successes, the trigger holds. Post-fix FPR: 0% on all three profiles.

3.5.3 Controller ablation (Jev vs. rule-only)

Jev serves as a probabilistic second opinion on soft decisions. Its agreement with deterministic rules varies by scenario:

Scenario	Rule fired	Jev confirmed	Jev override
infinite_loop	30/30	30 (100%)	0 (0%)
budget_bleed	30/30	1 (3%)	29 (97%)
dangerous	30/30	21 (70%)	9 (30%)
stall	25/30	0 (0%)	25 (100%)

Table 6: Jev agreement with deterministic rules on soft decisions.

3.5.4 Unseen failure modes

Three failure scenarios not present in the development set were generated to test generalization:

Scenario	Detection	Firing mechanism
permission_escalation	100%	Dangerous command regex
cascading_failure	100%	Retry count ≥ 3
silent_corruption	76.7%	Unverified-write streak

Table 7: Unseen failure modes (not used for tuning).

Silent corruption — an agent that writes many files in sequence without running any verification — was invisible to every trigger in the original configuration. The streak-based detector added during robustness testing catches most instances but not all: some randomized trials produce only 8–9 consecutive writes, below the threshold of 10. Full detection of this class requires monitoring the filesystem itself rather than the agent’s execution trace.

On the controller-ablation results: Jev overrides the deterministic rule on budget-bleed and stall in nearly all trials, returning CONTINUE where the rule recommends PAUSE. We interpret this cautiously. Two explanations are consistent with the data. First, Jev may correctly identify that the agent has not yet exhausted a threshold that warrants intervention (the MSS reports percentages, not absolute failure evidence). Second, Jev may systematically under-weight resource-consumption signals relative to convergence signals. Distinguishing these requires a task-success comparison: do agents allowed to continue under Jev’s judgment eventually succeed at a higher rate than agents paused by the rule alone? We have not run this experiment.

4 Discussion

4.1 What the State Engine provides that a watchdog cannot

The ablation results support a specific claim: failure modes defined by trajectory shape (non-convergence, stall, passive circling) require state that persists across turns. A single-event or simple-counter approach has no representation of “the error rate was low earlier and is high now,” or “the agent has read twelve files without modifying any.” These are properties of the sequence, not of any element in it.

The State Engine’s contribution is not compression for its own sake. The materialized state is smaller than the raw event history because it discards event content (what the agent actually read or wrote) and retains event shape (how often, in what order, with what error profile). This is sufficient for convergence and resource decisions but not for correctness decisions — a limitation we return to below.

4.2 What the current architecture cannot do

World-state corruption. An agent that writes destructive content to files without triggering any error, retry, or budget signal is invisible to the current system. The unverified-write streak heuristic catches the temporal pattern (many writes, no shell calls) but not the semantic one (the writes themselves are wrong). Detecting semantic corruption requires comparing actual filesystem state against an expected state — a Task Contract. This is architecturally distinct from execution monitoring and we have not implemented it.

Recovery quality. The recovery signal (three consecutive successes after errors) is a coarse heuristic. It does not verify that the successes are relevant to the original task, only that tool calls stopped failing. An agent that recovers from test failures by deleting the test file would produce a recovery signal.

Jev’s value is unquantified. We show that Jev agrees with rules on some scenarios and overrides on others, but we do not show that either behavior improves task outcomes. The policy resolver currently defaults to the rule when Jev’s confidence is below 0.5, which means Jev’s effective influence is limited to high-confidence overrides on soft decisions. Whether this is the right threshold — or whether Jev’s participation helps at all — requires an outcome-based evaluation we have not performed.

4.3 Threats to validity

1. **Circular validation.** Scenario generators and trigger rules were designed by the same authors with knowledge of each other. The held-out split controls for threshold overfitting but not for scenario-trigger co-adaptation.
2. **Simulated latency.** The stall scenario sets `started_at` retroactively rather than waiting in real time. Real agents may exhibit time-varying behavior (long compilations, network waits) that our synthetic clock does not capture.
3. **Single runtime.** Although the canonical schema normalizes event formats, we have only run the full pipeline end-to-end against Pi. The Codex and custom adapters are tested at the unit level, not against live agents.
4. **Jev API stability.** The TypeSafe API is an external dependency. Model version updates could shift Jev’s agreement rates without any change to our code.

5 Related considerations

The system intersects with several established areas. Circuit breakers in distributed systems provide the closest architectural analogue: a supervisor monitors a worker and cuts it off when failure signals exceed a threshold. Our contribution beyond this pattern is the state-materialization layer that enables trajectory-shape detection, and the two-tier authority model that separates deterministic safety from probabilistic judgment.

Sandboxing and permission systems (as in container-based agent execution) address the same safety surface from a different angle: restricting what the agent can do rather than observing

what it does. The two are complementary; a scope check in our trigger rules is a soft analogue of a filesystem sandbox, and could be replaced by one.

LLM-as-judge approaches evaluate agent outputs after completion. Our system differs in that it intervenes during execution, where the cost of a wrong decision is bounded by the remaining token budget rather than by the full task cost.

6 Future work

1. **Task Contract layer.** Define an expected-state specification (allowed paths, expected artifacts, forbidden operations) and compare against observed filesystem mutations. This addresses silent corruption directly.
2. **Outcome-based controller evaluation.** Run the same task with rule-only, Jev-gated, and no-controller configurations. Measure task success rate, token cost, and time-to-completion. This quantifies whether Jev’s participation improves outcomes.
3. **Real-trajectory validation.** Record event streams from real coding sessions, replay them through the control plane, and compare trigger decisions against human judgments of when intervention was warranted.
4. **Multi-agent coordination.** Extend the State Engine to track multiple agents sharing a filesystem, where one agent’s writes may invalidate another’s assumptions.

7 Conclusion

We presented Agent Control Plane, an external control layer that materializes cross-turn agent state from execution events and intervenes when the trajectory indicates non-convergence, resource exhaustion, dangerous operations, or boundary violations. The system achieves 100% detection and 0% false-positive rate on a held-out set of six synthetic scenarios with a frozen configuration. Ablation against a watchdog baseline confirms that trajectory-shape failure modes require persistent state, and robustness testing across four dimensions identifies specific boundaries — most notably the inability to detect world-state corruption from execution telemetry alone.

The results should be read as a characterization of what one particular architecture can and cannot detect under controlled conditions, not as evidence of general safety. The gap between execution monitoring and world-state verification is the clearest direction for future work.